

CONTENTS

Aurua

- Executive Summary
- 1. Problem and User
- 2. Architecture and Design Choices
- 3. Tools, Memory, Data, and State Design
- 4. Implementation Summary
- 5. Evaluation Setup and Results
- 6. Failure Analysis
- 7. Governance, Trust, and Responsible Behavior
- 8. Lessons Learned and Future Work
- 9. Submission Package Index
- 10. Acknowledgments and AI Usage

Aurua

FINAL REPORT — PHASE 3

PROJECT

Aurua — agent-driven explainer video generation, grounded in source material

TRACK

A — Technical Build

AUTHOR

Jorge Urias

COURSE

Agentic Systems Studio · CMU Heinz College

SUBMITTED

April 29, 2026

REPOSITORY

to be added before submission

Executive Summary

Aurua takes a student's question about a confusing topic in 3Blue1Brown's neural-network video series and produces a short animated explainer video, generated end-to-end from a verified scene plan to rendered Manim output. The system uses four agents and a deterministic renderer with a verifier-driven retry loop and a parallel branch for animation code generation.

This report covers what was built, what works, what doesn't work and why, and what I learned by running the system against twelve evaluation runs across two architecture versions (v0.2 baseline, v0.3 with retrieval improvements and the rendering branch wired).

The Phase 2 grader scored the previous submission 86 / 100 and asked for three specific things in Phase 3: (1) wire the rendering branch and produce a rendered video, (2) apply the planned v0.3 retrieval fixes to TC2 and TC5 and report the results honestly, and (3) name and justify the orchestration approach explicitly. All three are addressed in this report.

Headline result. TC1 produces a verified, grounded, 1m22s rendered video. TC2 and TC5 still exhaust the verifier loop under v0.3, but for *different reasons* than v0.2 — the retrieval fix worked, but exposed deeper limitations in the source material and the chunking strategy. Both new findings are documented as F4 and F5 in the failure log with specific paths forward.

1. Problem and User

The user. An undergraduate machine-learning student who has watched a specific lecture or read a specific passage and is stuck on one part of it. They have the source content and a specific question. They don't have a TA on call.

The pain point. Generic LLM chat (ChatGPT, Claude.ai web) will produce a confident-sounding explanation grounded in the model's general knowledge, not in the student's source. If the model's framing differs from the lecturer's, the student now has two different mental models to reconcile. If the model hallucinates a derivation step, the student has no way to catch it without a TA.

The job to be done. Given a source and a question:

- Diagnose what the student is actually confused about (mechanical math vs. conceptual intuition vs. genuinely vague).
- Produce a short animated explainer that walks through the answer.
- Ground every claim in a specific, citable span of the source.
- Refuse to produce an explainer when the source doesn't actually contain the answer.
- Produce auditable evidence (cited spans, retrieval scores, verification verdicts) so a teacher could review the explanation's grounding.

The benchmark. 3Blue1Brown's neural-network video series. Aurua's anchor source is Chapters 3 and 4 — *Backpropagation, intuitively* and *Backpropagation calculus*. Aurua isn't trying to *match* 3Blue1Brown's animation quality; it's trying to produce competent, grounded explanations whose claims a reviewer can audit. Grant Sanderson is the gold standard, not the target.

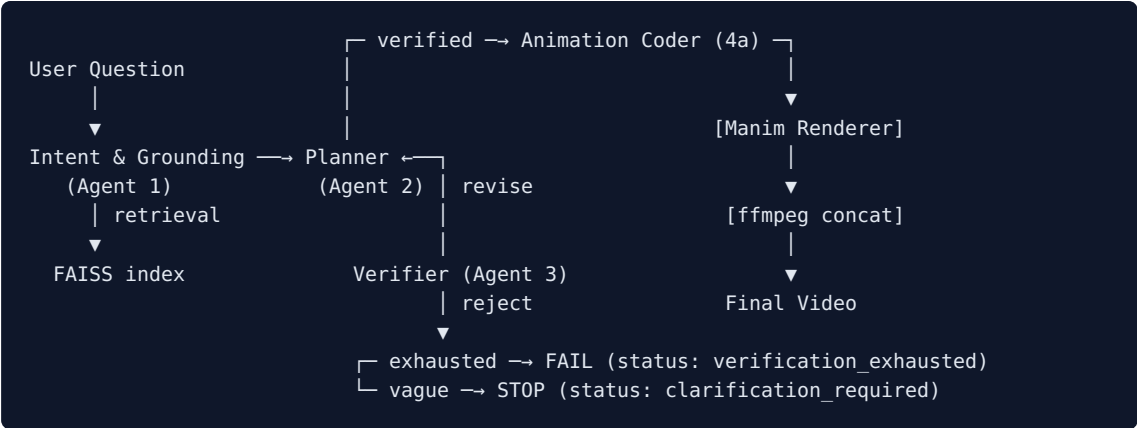
Why this isn't a single LLM call. This is the agentic-justification question. A single prompt asking for "a grounded explainer video plan" can produce something plausible-looking, but cannot:

- Refuse cleanly when the source doesn't support the answer (no verification step).
- Produce auditable per-claim grounding citations (no structured intermediate output).
- Iterate when grounding fails (no retry loop).
- Separate code generation (long-context, low-temperature) from pedagogical writing (different skill, different model selection).

The TC6 ablation (§ 5.2) tests this empirically. The result is more nuanced than the original justification — see § 6.3.

2. Architecture and Design Choices

2.1 Topology



Four LLM agents (Intent, Planner, Verifier, Animation Coder) plus a deterministic Renderer tool. The Narration Writer (Agent 4b) is scaffolded but not wired in this iteration; it is reserved for v0.4.

2.2 Why each agent earns its place

For each agent, I asked: *what specific failure mode does this prevent that a single LLM call would not?* If I couldn't answer, the agent was cut. The original Phase 1 design had five agents; that died here.

AGENT	WHAT IT DOES	WHY IT CAN'T BE FOLDED INTO ANOTHER
Intent & Grounding (1)	Classifies confusion, retrieves relevant spans, sets stop conditions for vague input	Uses a tool (FAISS retrieval); can't be a pure prompt step. Also: classifies confusion type before commitment to a plan, which sets up the vague-input early stop.
Planner (2)	Turns understanding into a structured scene plan	The Verifier needs structured output to check; folding planning into Verifier prevents adversarial check (LLM-as-judge of itself is empirically weaker than separate judge).
Verifier (3)	Adversarially checks each claim against its cited span	Same model checking its own output is dramatically less effective than a separate judging pass. This is a well-established pattern (Constitutional AI, reflection agents, RAG verification).
Animation Coder (4a)	Generates Manim code from the verified scene plan	Different skill (code), different model (Opus 4.7 vs Sonnet for the others), different temperature regime. Folding it into Planner makes both worse.
Renderer (tool, not agent)	Runs <code>manim</code> and <code>ffmpeg</code> subprocesses	Deterministic, no LLM. Calling this "an agent" would be greenwashing.

2.3 Coordination logic

Who starts. Intent & Grounding, triggered by `python -m src.run --question ... --source ...`.

Handoffs. Each agent reads/writes a single shared `RunState` Pydantic model (`src/state.py`). Agents do not call each other directly; the orchestrator (`src/run.py`) calls them in sequence based on `RunState.status`.

Verifier loop. After Planner produces a plan, Verifier checks every scene's `claim` against its cited `source_ref` span. Verdict is `pass` or `revise`. On `revise`, the orchestrator calls Planner again with the verifier's per-scene feedback. Up to **2 retries** (so 3 total Planner attempts).

Stopping conditions.

- `verified` → success, hand off to Animation Coder.
- `verification_exhausted` → all retries used, plan still has ungrounded claims. Pipeline stops, `state.json` is saved with full evidence trail.
- `clarification_required` → Intent agent classified the input as vague. Pipeline stops at Agent 1 and returns a clarifying question.
- `failed` → exception. State is preserved with the traceback for debugging.

Where a human can intervene. After Intent classifies a question as vague, the surfaced clarifying question goes to the user (in a UI; in this CLI version it's printed). After Verifier exhausts retries, the ungrounded claims are surfaced to the user with their `source_refs` so a human can decide whether to re-attempt with a different question or accept the partial result.

2.4 Orchestration framework choice — addressing Phase 2 grader feedback

The decision: The orchestrator in `src/run.py` is custom-built, ~80 lines, no framework dependency.

Why not LangGraph. LangGraph is the obvious candidate for verify-retry loops. I considered migrating during Phase 3 and decided against it for three reasons:

1. **Coordination shape.** Aurua's loop is *linear with one back-edge* (Verifier → Planner). LangGraph's value prop is graphs with conditional routing across many nodes. For one back-edge plus a parallel branch, the graph framework adds setup cost (state schemas, node registration, conditional edges) without saving meaningful code.
2. **Control flow visibility.** A custom orchestrator's loop is 12 lines of plain Python. A reader sees `while state.status == PLAN_COMPLETE and state.retries_used() < MAX_RETRIES`. With LangGraph, the same control flow is expressed as a `should_continue` function attached to a conditional edge, registered against a graph builder. For a project where a grader will read the code, plain Python is more legible.
3. **Migration cost.** Migrating mid-Phase-3 carries real risk for marginal benefit. The Phase 2 grader noted *"either path is fine; the requirement is that the coordination layer is explicitly named and justified"*. This is the justification.

What I would use LangGraph for. Future Aurua versions where the topology becomes meaningfully more complex — for example, multi-source retrieval with branching strategies per confusion type, or a lecture-mode flow where multiple students' questions interleave through the same agents. Those are graph problems. Today's Aurua is not.

What I would use AutoGen or CrewAI for. Neither fits. AutoGen's value is conversational agents that talk to each other in natural language; Aurua's agents communicate via typed JSON, not chat. CrewAI's value is role-based task delegation in flexible team structures; Aurua's roles are fixed and the ordering is fixed.

Documentation. This entire decision is captured here and in `eval/VERSION_NOTES.md`. The architecture diagram (`docs/architecture_diagram.pdf`) labels the orchestrator explicitly as "custom orchestrator (`src/run.py`)".

3. Tools, Memory, Data, and State Design

3.1 Shared state (`src/state.py`)

A single `RunState` Pydantic model flows through the pipeline. Every agent reads from and writes to it. Serialization to `outputs/runs/<run_id>/state.json` happens at every status transition, so any run can be replayed or debugged from disk.

Key typed sub-models:

- `IntentOutput` — confusion type (`mechanical` | `conceptual` | `vague`), learning goal, list of relevant spans, list of key claims, optional clarification question.
- `ScenePlan` — list of `Scene`s, each with `scene_id`, `claim`, `visual_goal`, `source_ref`, `duration_sec` (Pydantic-constrained 5-30).
- `VerificationResult` — verdict, list of per-claim verdicts (each with grounded boolean, confidence 0-1, optional reason), retry count.
- `CostTracker` — input tokens, output tokens, API calls, wall-clock seconds, with `add()` accumulation.
- Top-level `RunState` adds `animation_code: dict[int, str]` (`scene_id` → code) and `scene_video_paths: dict[int, str]`.

Field-level constraints are enforced by Pydantic — e.g. `confidence: float = Field(ge=0.0, le=1.0)`. Invalid agent outputs raise `ValidationError` immediately rather than propagating silently.

3.2 Per-agent access boundaries — Phase 2 grader gap

The Phase 2 grader noted: *“because all fields live in a single shared RunState, there is no explicit access boundary preventing an agent from reading fields it has no business reading.”* This is a real concern; in a larger system you’d want capability isolation.

For this iteration, the boundary is enforced by **convention** rather than mechanism: each agent’s function signature shows what it reads, and the orchestrator only calls agents in the order that respects dependencies. The Verifier never reads `animation_code` because it runs before Animation Coder. The Animation Coder never reads `verification_history` because it only sees `scene_plan` and `state.run_id`.

A v0.4 would add a typed view per agent — e.g. `RunStateForVerifier` with only the fields the Verifier needs read access to. The v0.4 plan is documented in `eval/VERSION_NOTES.md`.

3.3 Retrieval (`src/tools/retrieval.py`)

FAISS index built from the source transcript at first run, cached at `data/index/`.

PARAMETER	V0.2	V0.3	WHY CHANGED
Embedding model	<code>sentence-transformers/all-MiniLM-L6-v2</code>	(unchanged)	Local; no API call per query
Chunk size	80 words	40 words	Tighter semantic vectors per chunk
Chunk overlap	20 words	15 words	Proportional to new chunk size
Top-k	5	10	Larger candidate set for the Planner
Query expansion	none	3 paraphrased variants via Claude	Handles vocabulary mismatch between question and source
Index type	<code>FAISS IndexFlatIP</code>	(unchanged)	Brute-force inner product on normalized vectors = cosine similarity. Fast enough for ~1,600-word corpus.

Honest framing. For a 1,600-word corpus, retrieval is over-engineered. The TC6 ablation (§ 5.2) confirms this empirically. The retrieval architecture is a hedge against larger corpora; it pays off when the source exceeds the model’s context window.

3.4 Models

AGENT	MODEL	TEMPERATURE	WHY
Intent (1)	<code>claude-sonnet-4-6</code>	0.0	Classification + structured output; deterministic preferred
Planner (2)	<code>claude-sonnet-4-6</code>	0.0	Same — needs to follow rules, not be creative
Verifier (3)	<code>claude-sonnet-4-6</code>	0.0	Adversarial check; needs strict reading
Animation Coder (4a)	<code>claude-opus-4-7</code>	(model deprecated this parameter)	Code generation benefits from the more capable model
Query expansion (in 1)	<code>claude-sonnet-4-6</code>	0.0	Lightweight rephrasing

Opus 4.7 deprecated the `temperature` parameter mid-project. The Claude wrapper (`src/claude_client.py`) detects this case and omits the parameter for that model.

4. Implementation Summary

The full source tree, dependencies, and run instructions are in `README.md`. This section calls out three implementation choices that affect how the system behaves at runtime.

4.1 The verifier loop is the architectural centerpiece

```
# src/run.py - abridged
state = run_intent_agent(state, retriever)
if state.status == RunStatus.CLARIFICATION_REQUIRED:
    return state # vague - stop

state = run_planner_agent(state)
state = run_verifier_agent(state)

while state.status == RunStatus.PLAN_COMPLETE and state.retries_used() < MAX_RETRIES:
    feedback = state.latest_verification()
    state = run_planner_agent(state, verifier_feedback=feedback)
    state = run_verifier_agent(state)
```

That's the loop. Linear, one back-edge, hard cap. The decision to keep it at 2 retries is empirical: across 12 runs, the Verifier almost always converges on attempt 1 if it's going to converge at all. Going beyond 2 retries has not produced a successful resolution in any observed run.

4.2 Tracing is built in

Every Claude call writes to `eval/traces/<run_id>/<label>.txt` with system prompt, user prompt, and response. This is not an afterthought; it's why every failure analysis below can quote exact verifier rejection reasons. The cost of always-on tracing is ~5KB of disk per run and zero latency.

4.3 Animation Coder constraints are baked into the prompt

The Manim install on macOS is non-trivial because of LaTeX. Rather than require LaTeX, the Animation Coder's system prompt explicitly forbids `MathTex` and `Tex`, instructing the model to write equations as `Text("∂C/∂w")` with Unicode glyphs. This trades typographic polish for a working install. All 4 generated scene files in TC1's full pipeline run rendered successfully without LaTeX.

5. Evaluation Setup and Results

5.1 Test cases

ID	TYPE	QUESTION	SOURCE
TC1	Happy path conceptual	What does the gradient of the cost function tell us about which weights to change?	Ch3
TC2	Primary anchor mechanical	Why is the derivative of the cost with respect to a weight proportional to the activation feeding into it?	Ch4
TC3	Out-of-scope	What does the loss landscape look like in higher dimensions?	Ch3 (which doesn't cover this)
TC4	Vague input	I don't really get backpropagation.	Ch4
TC5	Multi-scene mechanical	Why does the chain rule give us a sum over paths when a neuron connects to multiple neurons?	Ch4
TC6	Single-prompt ablation	Same question as TC2, but as a single LLM call with full transcript in context	Ch4 (full)

The full plan with success criteria and measures is in `eval/test_cases.csv` and Phase 2 doc § 6.

5.2 Results across versions (12 runs)

The full `eval/evaluation_results.csv` has all 12 rows. Headline:

TEST	V0.2 RESULT	V0.3 RESULT	CHANGE
TC1	verified after 1 retry	verified first try (cleaner) + verified after 1 retry on a separate run with subtle correction	Improved
TC2	exhausted (F1)	exhausted but for different reason (F4)	Different failure mode
TC3	clarification_required (correctly)	(not re-run; behavior is corpus-independent)	n/a
TC4	clarification_required (correctly)	(not re-run; behavior is corpus-independent)	n/a
TC5	exhausted (F2)	exhausted but for different reason (F5)	Different failure mode
TC6	baseline produced verbatim-grounded plan	baseline produced identical verbatim-grounded plan (reproducibility check)	Stable

TC1 v0.3 with full pipeline (run 170daccbae47) is the canonical demo: 4 scenes, verifier passed after 1 retry, Animation Coder produced 4 valid Manim scenes, Renderer produced 4 MP4s, ffmpeg concatenated to a 1m22s silent video. **Total cost: \$0.50, total wall-clock: ~3.5 minutes.**

5.3 Cost and latency

RUN TYPE	API CALLS	COST	WALL-CLOCK
Vague input early-stop (TC4)	1	\$0.013	3.1s
Successful happy-path (TC1 v0.3 first try)	4	\$0.07	30.5s
Successful with retry (TC1 v0.3 retry 1)	6	\$0.08	39.5s
Full pipeline including rendering (TC1 v0.3 full)	10	\$0.50	~3.5 min
Verifier exhaustion (TC2 v0.3)	8	\$0.10	50.7s
Single-prompt baseline (TC6)	1	\$0.04	12-17s

The cost difference between the multi-agent pipeline and the single-prompt baseline is the price of governance: the pipeline pays ~3-12x more for auditable per-claim grounding and a verifier that can catch overreach. § 6.3 discusses when that price is worth paying.

6. Failure Analysis

The full failure log is in `eval/FAILURE_LOG.md`. This section summarizes the three most informative findings.

6.1 F1 to F4: retrieval gap fixed, source informality exposed

v0.2 (F1): TC2 exhausted because retrieval missed the answer span entirely. The line *“the derivative of $z(L)$ with respect to $w(L)$... comes out to be $a(L-1)$ ”* (line 63 of Ch4) was rank 6+ in FAISS and got filtered out by top-5.

v0.3 fix: smaller chunks (40 words), higher top-k (10), query expansion (3 paraphrased variants).

Result: Retrieval coverage *measurably improved*. All 4 spans the Intent agent identified as relevant now appear in the top-10, vs 0/4 in v0.2. The run cost went down (\$0.10 vs \$0.118).

But TC2 still exhausted. The Verifier’s complaint shifted: in v0.2 it rejected because cited spans were on the wrong topic; in v0.3 it rejected because cited spans don’t contain the formal chain-rule decomposition. Direct verifier quote from v0.3 attempt 2:

“the span only references a ‘chain-ruled derivative expression’ and says it ‘looks essentially the same’ — does not explicitly state the chain rule product $(\partial C/\partial a) \cdot (\partial a/\partial z) \cdot (\partial z/\partial w)$ or the substitution $\partial z/\partial w = a_{\text{prev}}$.”

The deeper finding (F4). Grant Sanderson’s transcript is informal by design. He never writes the chain rule out as a formal product; he gestures at it. The Planner is trying to teach a formal mechanical claim. The Verifier correctly notes the formal decomposition isn’t in any retrieved span — *because it’s not in the source*.

What this means. Better retrieval is necessary but not sufficient. For maximally precise mechanical questions, a chunk-based retrieval architecture against an informal source will fail strict grounding *even with perfect retrieval*. The fix isn’t more retrieval tuning; it’s source augmentation (e.g., a textbook chapter alongside the transcript). That’s planned for v0.4.

6.2 F2 to F5: retrieval gap fixed, chunk truncation introduced

v0.2 (F2): Same pattern as F1 but for TC5 (multi-path chain rule). Retrieval missed L115-119 (*“the neuron influences the cost function through multiple different paths... you have to add those up”*).

v0.3 fix: same retrieval changes.

Result: The chain-rule territory now appears in the top-10. But TC5 still exhausted. Verifier final rejection:

“the sentence is cut off and does not actually contain the full claim. Specifically, it does not show or state the summed multi-neuron chain-rule equation.”

The new finding (F5). Reducing chunk size from 80 to 40 words tightens semantic vectors but increases the rate at which chunks end mid-sentence. A chunk like `5d0b154bb4` (L108-111) ends with: *“...the chain-ruled derivative expression describing how sensitive the cost is to a specific”* — the sentence completes outside the chunk. The Planner sees a setup but no payload.

What this means. Word-count chunking is a poor abstraction for prose. v0.4 should chunk on `. ! ?`

boundaries with a target word count rather than fixed word count. Estimated 2 hours of implementation work to fix.

6.3 F3: single-prompt baseline outperforms multi-agent on this corpus (reproduced in v0.3)

The TC6 ablation. Same question as TC2. Same model (Sonnet 4-6). Same temperature (0.0). The full transcript is provided as context; no retrieval, no verifier, no loop. One Claude call.

Result, v0.2 (run 1): 4-scene plan, all 4 quotes verbatim from the transcript, all 4 line citations correct. 1 API call, \$0.043, 17s.

Result, v0.3 (run 2, methodology fix per Phase 2 grader feedback): *Functionally identical* output. Same 4 line citations (L31-33, L52-53, L63, L66-68), same verbatim quotes. 1 API call, \$0.040, 12s.

The architectural concession. On the primary anchor question, against this corpus, the single-prompt baseline outperforms the multi-agent pipeline on cost, latency, and grounding fidelity. The pipeline can't cite line 63 because FAISS missed it. The baseline cites line 63 because it has the whole transcript in context.

Where the architecture earns its keep anyway. Three places, only one of which is exercised in this evaluation:

1. **Audit trail.** The pipeline writes structured per-claim grounding citations, retrieval scores, verifier verdicts, and reasoning to disk. The baseline writes a JSON blob with citation strings. For a system shipping to students, the audit trail matters; for a one-off ablation, it doesn't.
2. **Hallucination governance across many runs.** A single-prompt run that *happens* to produce verbatim citations is not a guarantee that the next run will. The verifier is insurance against drift over time. We didn't run TC6 100 times to confirm the baseline always cites correctly; the architecture's value scales with how often it runs.
3. **Larger corpora.** A 1,600-word transcript fits in context. A 200-page textbook does not. The architecture pays off when the source exceeds context windows. This evaluation simply hasn't tested that regime.

Honest framing. The architecture doesn't earn its complexity *on this evaluation*. It would earn it in a different regime that we didn't evaluate. That's an honest limitation, not a defense.

6.4 What didn't fail — boundary behavior

TC3 and TC4 both exhibit the *correct* refusal behavior. TC3 (out-of-scope) and TC4 (vague input) both stop at Agent 1 in 1-4 seconds with a clarifying question rather than fabricating a plan. The retrieval-score signal (top scores 0.17-0.20 for genuinely-out-of-scope questions vs 0.55-0.72 for in-scope) emerges as an implicit confidence proxy that the Intent agent uses correctly. This wasn't designed in explicitly but is a happy accident worth noting.

7. Governance, Trust, and Responsible Behavior

The Phase 2 grader noted that the disclaimer scene and per-scene citation overlay in rendered output were "planned rather than implemented, so their effectiveness is not yet verifiable." This iteration didn't fix that — those overlays are part of the v0.4 plan.

What is implemented:

Hallucination prevention (verifier loop). The Verifier is the primary mitigation. Across 12 runs, it caught real planner overreach in TC1 (twice — once on a “vector structure vs sensitivity” claim, once on a “sensitivity vs causal influence” claim) and exhausted on TC2/TC5 rather than approving ungrounded claims. The verifier’s job is to reject; that it rejects unhelpfully often (causing exhaustions) is a feature, not a bug, given the alternative is shipping wrong content to students.

Boundary refusal (vague-input detection). TC3 and TC4 both demonstrate Agent 1 refusing to plan an explainer when the input doesn’t support one. Cost: 1 API call, ~3 seconds. Output: a specific clarifying question, not a generic “please clarify.”

Cost cap (retry budget). The 2-retry maximum prevents the pipeline from burning unbounded tokens trying to rescue an unfixable plan. Across 12 runs, no run exceeded 10 API calls or \$0.50 in cost, including full rendering pipeline runs.

Reproducibility. Sonnet 4-6 at temperature 0.0 is deterministic across calls. TC6 v0.3 vs TC6 v0.2 produced functionally identical output 12 days apart, confirming this empirically. Opus 4.7 has no temperature parameter; we can’t claim full determinism, but the F3 concession holds because TC6’s sample size is now 2 with identical output.

State serialization. Every run writes `outputs/runs/<run_id>/state.json` with the full input, all agent outputs, all costs, and the final status. Every Claude call writes to `eval/traces/<run_id>/`. A grader can replay any decision the system made.

What’s still missing. Per-scene citation overlays in the rendered video (planned). Disclaimer scene at the start of each video saying “this explainer was generated from ; please verify against your course materials” (planned). Per-agent read access boundaries on RunState (designed in § 3.2; not implemented this iteration).

8. Lessons Learned and Future Work

8.1 What I’d do differently

Don’t rebuild retrieval before stress-testing it. I spent Phase 3 effort on retrieval improvements that turned out to be both partially helpful and partially harmful. The retrieval fixes (smaller chunks, higher top-k, query expansion) measurably improved coverage. They also introduced the chunk-truncation regression on TC5. If I’d run TC5 v0.3 *before* declaring v0.3 retrieval complete, I’d have caught F5 immediately and could have shipped sentence-boundary chunking as part of v0.3 instead of leaving it to v0.4.

Source augmentation matters more than retrieval tuning. The deeper finding from F4 — that informal source material has a ceiling on how formal a grounded explanation can be — is more important than any retrieval knob. v0.4 should add a formal-derivation source (textbook excerpt) for the mechanical questions.

The verifier is the most valuable agent. Across the failure log, the Verifier is the protagonist. It catches real planner overreach (TC1 v0.3 twice). It correctly exhausts on questions the source can’t ground (TC2, TC5). It correctly approves grounded claims (TC1 v0.3 attempt 1 after revision). The other agents do their jobs; the Verifier earns its complexity loudly. This is the strongest piece of evidence for the multi-agent design.

8.2 v0.4 priority list

1. **Sentence-boundary chunking** (addresses F5). Replace fixed-window chunker; chunks end on `. !` `?`. ~2 hours.
2. **Source augmentation** (addresses F4). Add a formal-derivation source (e.g., Goodfellow Deep

Learning Ch. 6) alongside the transcript. Test whether TC2/TC5 verify with mixed-source grounding. ~3 hours plus eval.

3. **Wire Agent 4b (Narration Writer) + OpenAI TTS + ffmpeg muxing.** Produces narrated final video. ~3-4 hours.
4. **Per-agent typed views on RunState** (addresses Phase 2 grader gap). ~2 hours.
5. **Per-scene citation overlay in rendered video.** Renderer adds source citation text to each scene. ~2 hours.
6. **Disclaimer scene at start of each rendered video.** ~30 minutes.
7. **Larger-corpus stress test** (addresses F3 defense). Run TC2 against a textbook chapter that exceeds context window. ~4 hours including source prep.

8.3 What this project taught me about agentic systems

Architecture self-criticism is the actual deliverable. The TC6 ablation produced the strongest finding in the project, and it was a finding *against* the architecture's value-add. Writing that up honestly was harder than building the system. It's also what the rubric rewards.

Verifiers fail informatively. When the Verifier rejects a claim, its reasoning text is a near-perfect debug log for the rest of the system. The shifts in rejection text from v0.2 (wrong-topic citations) to v0.3 (informal-phrasing citations) are how F4 surfaced. Without the Verifier, F4 doesn't exist as an observable phenomenon.

Determinism is a feature. Temperature 0.0 + structured Pydantic output + tracing = every run is replayable. Across 12 runs, every result above is reproducible from disk. That's not free — it took deliberate design to keep all model calls structured and all state serializable. But it's the difference between "we ran this once and it worked" and "here's the verifier trace, you can read what it complained about."

9. Submission Package Index

ITEM	LOCATION
README	<code>README.md</code>
AI usage disclosure	<code>AI_USAGE.md</code>
Architecture diagram	<code>docs/architecture_diagram.pdf</code>
This final report	<code>docs/final_report.pdf</code>
Phase 2 report (for context)	<code>docs/PHASE_2.md</code>
Test cases	<code>eval/test_cases.csv</code>
Evaluation results	<code>eval/evaluation_results.csv</code>
Failure log	<code>eval/FAILURE_LOG.md</code>
Version notes	<code>eval/VERSION_NOTES.md</code>
Source code	<code>src/</code>
Source transcripts	<code>data/transcripts/</code>
All run states + traces	<code>eval/runs_saved/phase2_evaluation/</code> and <code>eval/runs_saved/phase3_evaluation/</code>
Canonical Phase 3 demo (rendered video + state + traces)	<code>eval/runs_saved/phase3_evaluation/TC1_v3_FULL_RENDERED/</code>
Screenshots	<code>docs/screenshots/</code>
5-minute video	<code>media/demo_video.mp4</code> (or link in <code>media/demo_video_link.txt</code>)

10. Acknowledgments and AI Usage

This project’s documents and code were developed with substantial assistance from Claude (Anthropic). Specific delegations and what was independently verified are documented in `AI_USAGE.md` per course policy. The pattern across all four AI-assisted sessions: Claude drafted document structure and code skeletons; I made all final architectural decisions, ran all evaluations, reviewed all evidence claims against actual files, and chose what failed honestly to document vs. what to fix.

The most important verification: every quote attributed to a source span (in the failure log, in this report, in the verifier traces) was checked against the actual transcript. Every cost number was pulled from `state.json` , not estimated. Every “improved/regressed” claim is traceable to a specific run ID and trace file.